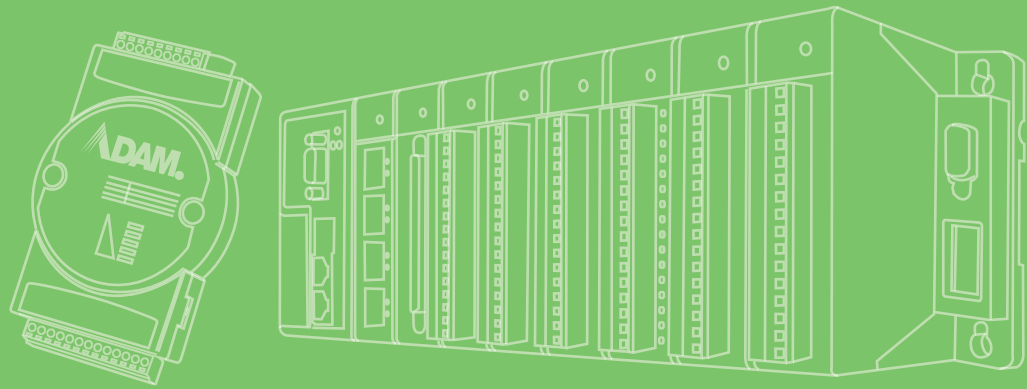




Programming Guide

ICAM-300 Series

Industrial AI Camera

Copyright

The documentation and the software included with this product are copyrighted 2026 by Advantech Co., Ltd. All rights are reserved. Advantech Co., Ltd. reserves the right to make improvements in the products described in this manual at any time without notice. No part of this manual may be reproduced, copied, translated, or transmitted in any form or by any means without the prior written permission of Advantech Co., Ltd. The information provided in this manual is intended to be accurate and reliable. However, Advantech Co., Ltd. assumes no responsibility for its use, nor for any infringements of the rights of third parties that may result from its use.

Acknowledgments

Intel and Pentium are trademarks of Intel Corporation.

Microsoft Windows and MS-DOS are registered trademarks of Microsoft Corp.

All other product names or trademarks are properties of their respective owners.

Product Warranty (2 years)

Advantech warrants the original purchaser that each of its products will be free from defects in materials and workmanship for two years from the date of purchase.

This warranty does not apply to any products that have been repaired or altered by persons other than repair personnel authorized by Advantech, or products that have been subject to misuse, abuse, accident, or improper installation. Advantech assumes no liability under the terms of this warranty as a consequence of such events.

Because of Advantech's high quality-control standards and rigorous testing, most customers never need to use our repair service. If an Advantech product is defective, it will be repaired or replaced free of charge during the warranty period. For out-of-warranty repairs, customers will be billed according to the cost of replacement materials, service time, and freight. Please consult your dealer for more details.

If you believe your product to be defective, follow the steps outlined below.

1. Collect all the information about the problem encountered. (For example, CPU speed, Advantech products used, other hardware and software used, etc.) Note anything abnormal and list any onscreen messages displayed when the problem occurs.
2. Call your dealer and describe the problem. Please have your manual, product, and any helpful information readily available.
3. If your product is diagnosed as defective, obtain a return merchandise authorization (RMA) number from your dealer. This allows us to process your return more quickly.
4. Carefully pack the defective product, a completed Repair and Replacement Order Card, and a proof of purchase date (such as a photocopy of your sales receipt) into a shippable container. Products returned without a proof of purchase date are not eligible for warranty service.
5. Write the RMA number clearly on the outside of the package and ship the package prepaid to your dealer.

Declaration of Conformity

CE

This product has passed the CE test for environmental specifications when shielded cables are used for external wiring. We recommend the use of shielded cables. This type of cable is available from Advantech. Please contact your local supplier for ordering information.

Test conditions for passing also include the equipment being operated within an industrial enclosure. In order to protect the product from damage caused by electrostatic discharge (ESD) and EMI leakage, we strongly recommend the use of CE-compliant industrial enclosure products.

FCC Class A

Note: This equipment has been tested and found to comply with the limits for a Class A digital device, pursuant to part 15 of the FCC Rules. These limits are designed to provide reasonable protection against harmful interference when the equipment is operated in a commercial environment. This equipment generates, uses, and can radiate radio frequency energy and, if not installed and used in accordance with the instruction manual, may cause harmful interference to radio communications. Operation of this equipment in a residential area is likely to cause harmful interference in which case the user will be required to correct the interference at their own expense.

Technical Support and Assistance

1. Visit the Advantech website at www.advantech.com/support to obtain the latest product information.
2. Contact your distributor, sales representative, or Advantech's customer service center for technical support if you need additional assistance. Please have the following information ready before calling:
 - Product name and serial number
 - Description of your peripheral attachments
 - Description of your software (operating system, version, application software, etc.)
 - A complete description of the problem
 - The exact wording of any error messages

Safety Precaution - Static Electricity

Follow these simple precautions to protect yourself from harm and the products from damage.

- To avoid electrical shock, always disconnect the power from the PC chassis before manual handling. Do not touch any components on the CPU card or other cards while the PC is powered on.
- Disconnect the power before making any configuration changes. A sudden rush of power after connecting a jumper or installing a card may damage sensitive electronic components.

Safety Instructions

1. Read these safety instructions carefully.
2. Retain this user manual for future reference.
3. Disconnect the equipment from all power outlets before cleaning. Use only a damp cloth for cleaning. Do not use liquid or spray detergents.
4. For pluggable equipment, the power outlet socket must be located near the equipment and easily accessible.
5. Protect the equipment from humidity.
6. Place the equipment on a reliable surface during installation. Dropping or letting the equipment fall may cause damage.
7. The openings on the enclosure are for air convection. Protect the equipment from overheating. Do not cover the openings.
8. Ensure that the voltage of the power source is correct before connecting the equipment to a power outlet.
9. Position the power cord away from high-traffic areas. Do not place anything over the power cord.
10. All cautions and warnings on the equipment should be noted.
11. If the equipment is not used for a long time, disconnect it from the power source to avoid damage from transient overvoltage.
12. Never pour liquid into an opening. This may cause fire or electrical shock.
13. Never open the equipment. For safety reasons, the equipment should be opened only by qualified service personnel.
14. If any of the following occurs, have the equipment checked by service personnel:
 - The power cord or plug is damaged.
 - Liquid has penetrated the equipment.
 - The equipment has been exposed to moisture.
 - The equipment is malfunctioning, or does not operate according to the user manual.
 - The equipment has been dropped and damaged.
 - The equipment shows obvious signs of breakage.
15. Do not leave the equipment in an environment with a storage temperature of below -20 °C (-4 °F) or above 60 °C (140 °F) as this may damage the components. The equipment should be kept in a controlled environment.
16. CAUTION: Batteries are at risk of exploding if incorrectly replaced. Replace only with the same or equivalent type as recommended by the manufacturer. Discard used batteries according to the manufacturer's instructions.
17. In accordance with IEC 704-1:1982 specifications, the sound pressure level at the operator's position does not exceed 70 dB (A).
18. Install the system only in area with restricted access.
19. DISCLAIMER: These instructions are provided according to IEC 704-1 standards. Advantech disclaims all responsibility for the accuracy of any statements contained herein.

Wichtige Sicherheitshinweise

1. Bitte lesen sie Sich diese Hinweise sorgfältig durch.
2. Heben Sie diese Anleitung für den späteren Gebrauch auf.
3. Vor jedem Reinigen ist das Gerät vom Stromnetz zu trennen. Verwenden Sie Keine Flüssig-oder Aerosolreiniger. Am besten dient ein angefeuchtetes Tuch zur Reinigung.
4. Die NetzanschlusBsteckdose soll nahe dem Gerät angebracht und leicht zugänglich sein.
5. Das Gerät ist vor Feuchtigkeit zu schützen.
6. Bei der Aufstellung des Gerätes ist auf sicheren Stand zu achten. Ein Kippen oder Fallen könnte Verletzungen hervorrufen.
7. Die Belüftungsöffnungen dienen zur Luftzirkulation die das Gerät vor überhitzung schützt. Sorgen Sie dafür, daB diese Öffnungen nicht abgedeckt werden.
8. Beachten Sie beim. AnschluB an das Stromnetz die AnschluBwerte.
9. Verlegen Sie die NetzanschlusBleitung so, daB niemand darüber fallen kann. Es sollte auch nichts auf der Leitung abgestellt werden.
10. Alle Hinweise und Warnungen die sich am Geräten befinden sind zu beachten.
11. Wird das Gerät über einen längeren Zeitraum nicht benutzt, sollten Sie es vom Stromnetz trennen. Somit wird im Falle einer Überspannung eine Beschädigung vermieden.
12. Durch die Lüftungsöffnungen dürfen niemals Gegenstände oder Flüssigkeiten in das Gerät gelangen. Dies könnte einen Brand bzw. elektrischen Schlag auslösen.
13. Öffnen Sie niemals das Gerät. Das Gerät darf aus Gründen der elektrischen Sicherheit nur von autorisiertem Servicepersonal geöffnet werden.
14. Wenn folgende Situationen auftreten ist das Gerät vom Stromnetz zu trennen und von einer qualifizierten Servicestelle zu überprüfen:
 15. Netzkabel oder Netzstecker sind beschädigt.
 16. Flüssigkeit ist in das Gerät eingedrungen.
 17. Das Gerät war Feuchtigkeit ausgesetzt.
18. Wenn das Gerät nicht der Bedienungsanleitung entsprechend funktioniert oder Sie mit Hilfe dieser Anleitung keine Verbesserung erzielen.
19. Das Gerät ist gefallen und/oder das Gehäuse ist beschädigt.
20. Wenn das Gerät deutliche Anzeichen eines Defektes aufweist.
21. **VORSICHT:** Explosionsgefahr bei unsachgemaben Austausch der Batterie.Ersatz nur durch denselben oder einem vom Hersteller empfohlene-mahnlichen Typ. Entsorgung gebrauchter Batterien navh Angaben des Herstellers.
22. **ACHTUNG:** Es besteht die Explosionsgefahr, falls die Batterie auf nicht fachmännische Weise gewechselt wird. Verfangen Sie die Batterie nur gleicher oder entsprechender Type, wie vom Hersteller empfohlen. Entsorgen Sie Batterien nach Anweisung des Herstellers.
23. Der arbeitsplatzbezogene Schalldruckpegel nach DIN 45 635 Teil 1000 beträgt 70dB(A) oder weiger.

Haftungsausschluss: Die Bedienungsanleitungen wurden entsprechend der IEC-704-1 erstellt. Advantech lehnt jegliche Verantwortung für die Richtigkeit der in diesem Zusammenhang getätigten Aussagen ab.

Contents

Chapter 1	Introduction.....	1
1.1	Introduction	2
1.2	Solution Features	2
	Table 1.1: Solution Features	2
1.3	Limitations	2
Chapter 2	ICAM-300 Series SDK.....	5
2.1	ICAM-300 Series Web Service	6
2.2	ICAM-300 Series Web Service and CamNavi2 SDK Operation	7
Chapter 3	Operating Workflow.....	9
3.1	Open/Close Camera Flow	10
3.2	Image Acquisition	11
3.3	Image Trigger Flow	12
	3.3.1 Software Trigger	13
3.4	Lens Focusing Control	14
	3.4.1 Digital Input flow.....	15
Chapter 4	CamNavi2 SDK API Manual	17
	Table 4.1: Module CamNavi2	18
	Table 4.2: iCam500 Components.....	18
4.1	Requirements.....	19
	4.1.1 Basic	19
4.2	Module CamNavi2.....	19
	4.2.1 Class CamNavi2	19
4.3	Module iCam500	26
	4.3.1 Class iCam500.....	26
	4.3.2 enum CameraMode	35
4.4	Module Acquisition	35
4.5	Module Image	36
	4.5.1 Class CImage	36
4.6	Module Focus.....	41
	4.6.1 Class Focus	41
4.7	Module Lighting.....	43
	4.7.1 Class Lighting	43
4.8	Module Board.....	44
	4.8.1 Class Board	44
4.9	Module Digitalio.....	44
	4.9.1 Digital I/O Code Slice.....	44
	4.9.2 Class DigitalIO	45
4.10	Module Device	47
	4.10.1 Class Network.....	47
	4.10.2 Class Firmware	47
4.11	Exceptions.....	48
	Table 4.3: API Support Table	48
Chapter 5	CamNavi2 SDK Python Examples....	53

5.1	Grab Frame Continuous	54
5.2	Grab Frames by Software Trigger	56
5.3	Digital Input Processing	59
5.4	Lens Focusing Control	61
5.5	Lighting Control.....	64
5.6	Get Image from Video10	65

Chapter 1

Introduction

1.1 Introduction

The Advantech CAMNavi SDK offers the tools which provide developers/ ISVs to operate ICAM-500 series Industrial AI cameras and deploy the vision AI APP. based on NVIDIA Jetpack. The CAMNavi SDK uses Python language by default and is better adapted to image acquisition and AI algorithm integration.

1.2 Solution Features

- The Advantech CAMNavi SDK and Qualcomm QCS6490 are provided with the BSP for the ICAM-300 series. Users must run the CAMNavi SDK on the ICAM-300 series.
- To use CAMNavi SDK

Table 1.1: Solution Features

System requirement	ICAM-300	ICAM-540
OS	Yocto	Linux Ubuntu 20.04
Language	Python 3.10	Python 3.8

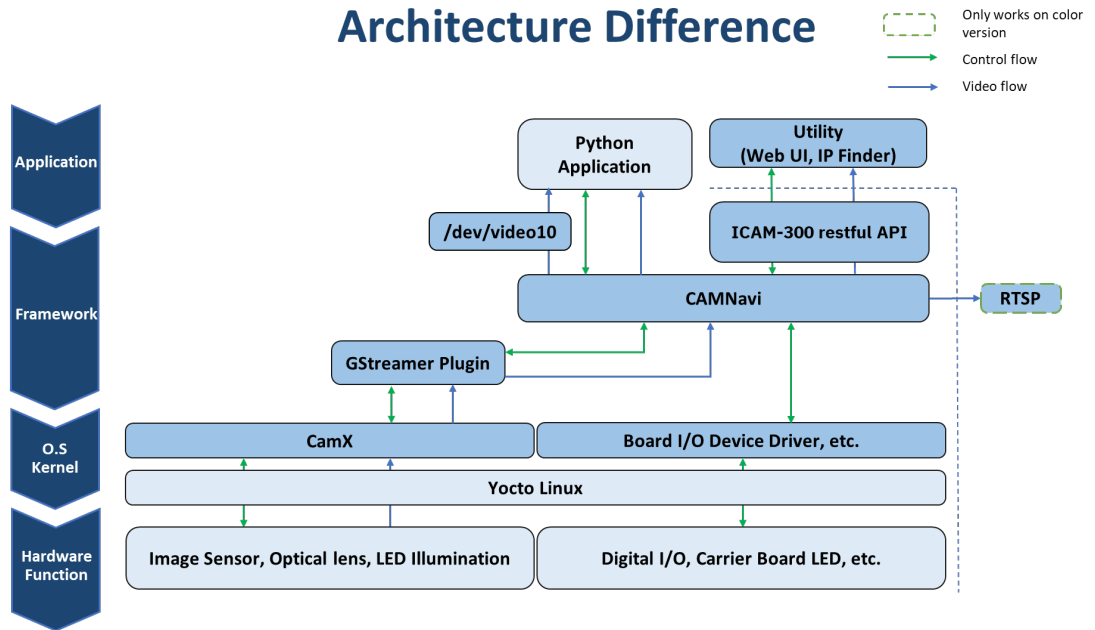
1.3 Limitations

- The CAMNavi SDK only operates on ICAM-300 series.
- Disable the web service before programming the ICAM-300 series with the CAMNavi SDK, because the web service controls the video bus and I/O of the ICAM-300 series for the web utility. Follow the instructions in Chapter 3 to disable the web service.

Chapter 2

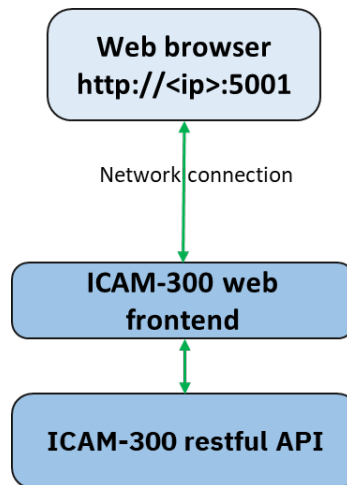
ICAM-300 Series SDK

Architecture Difference



2.1 ICAM-300 Series Web Service

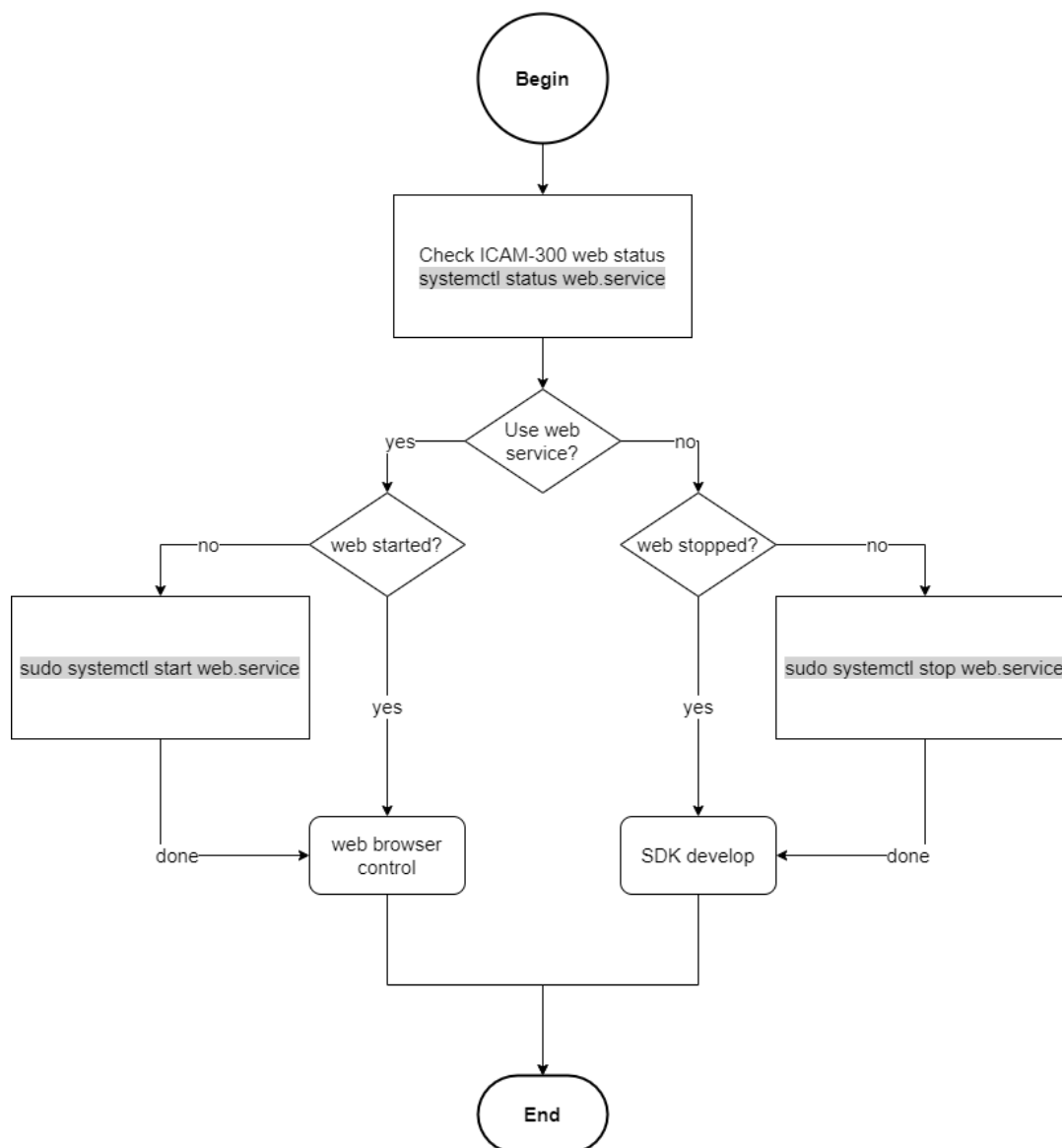
ICAM-500 series can use a WWW browser (Chrome and Firefox preferred) interface to provide its web service. The service uses ICAM-300 series restful APIs to control functions of the ICAM-300 series devices.



2.2 ICAM-300 Series Web Service and CamNavi2 SDK Operation

The ICAM-300 series web service uses the CAMNavi2 SDK to control ICAM-300 series devices. Only one instance can control the ICAM-300 series functions at a time. If developers use the SDK to write Python applications, the web service should be turned off to release control of the SDK.

The flowchart below shows the decision path for starting or stopping the web service.



Chapter 3

Operating Workflow

To utilize the ICAM-300 series Python SDK, the web service must be disabled.

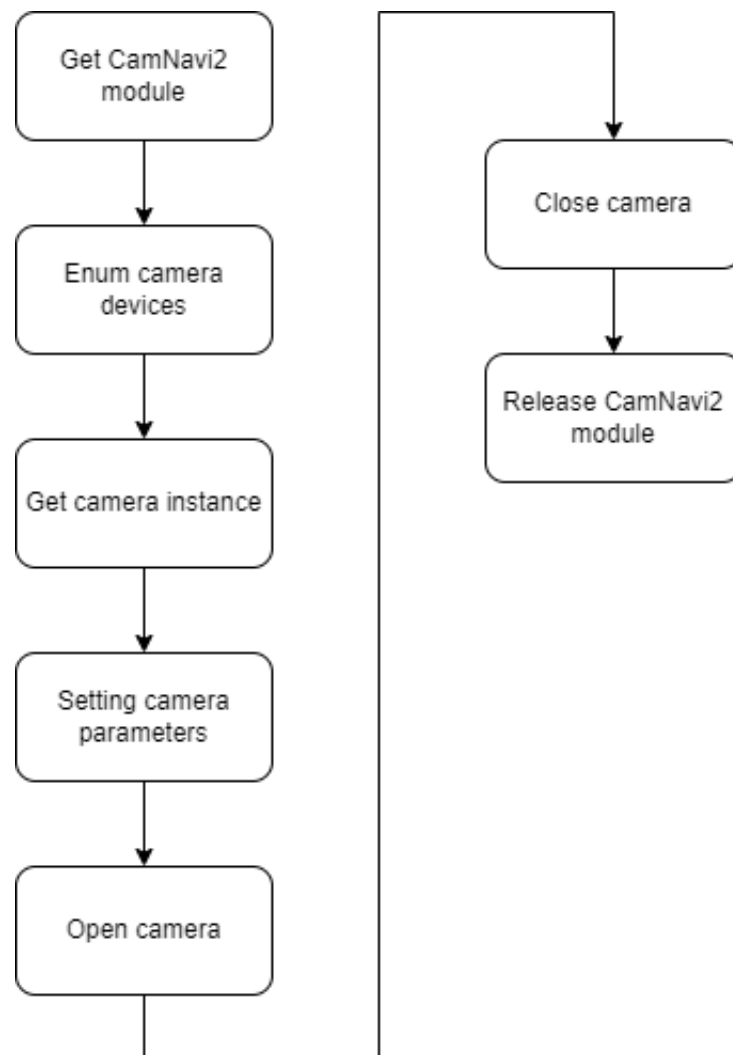
1. Temporarily stop the web service.

```
$ sudo systemctl stop web.service
```

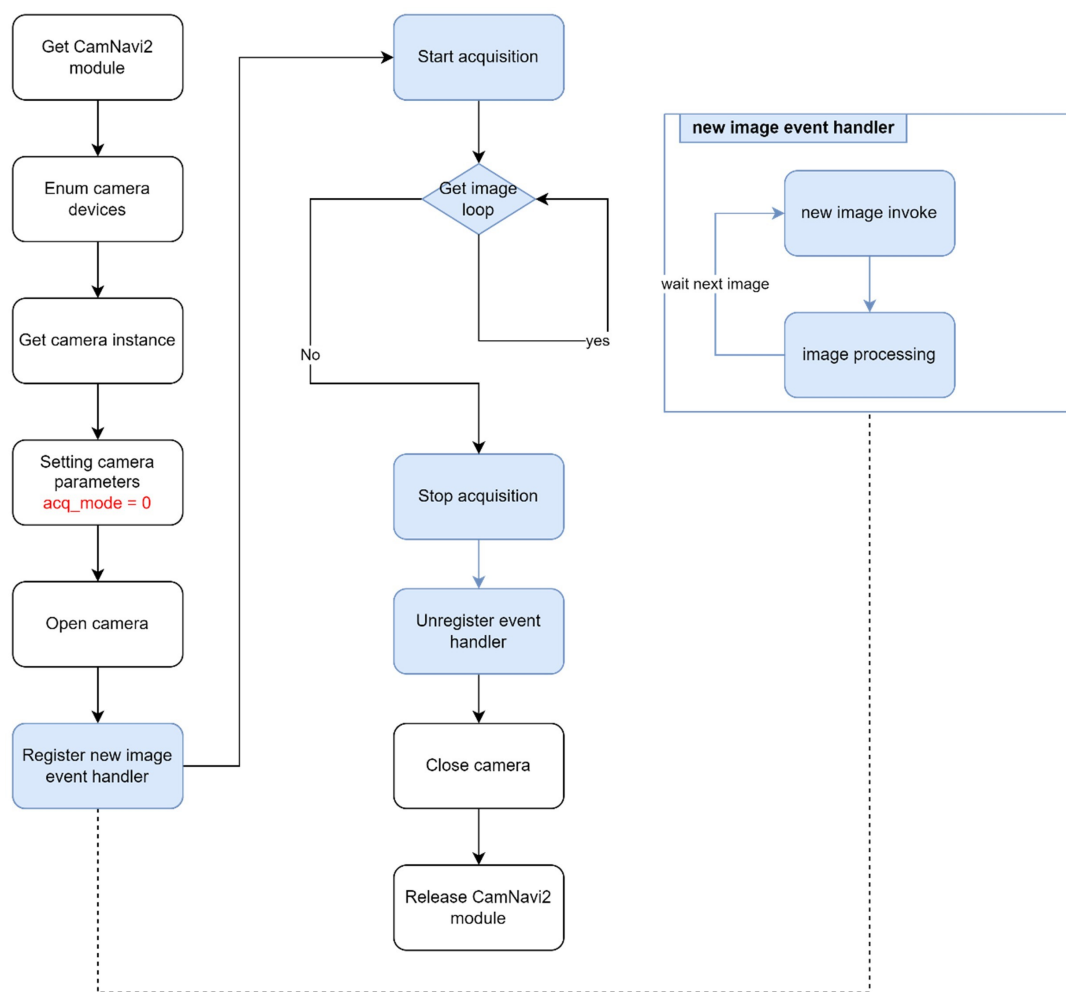
2. Permanently stop the iCAM-300 web service.

```
$ sudo systemctl stop web.service  
$ sudo systemctl disable web.service  
$ sudo systemctl disable autoui.service
```

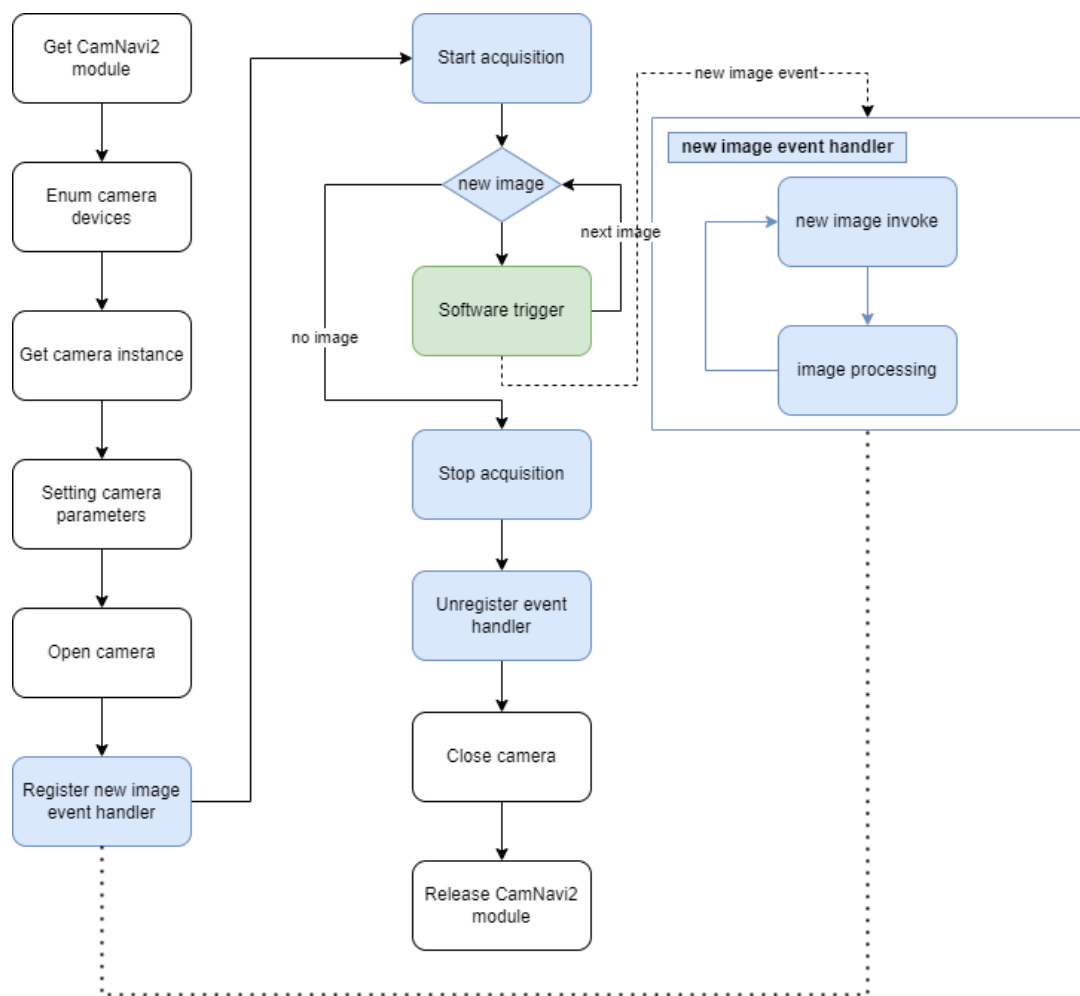
3.1 Open/Close Camera Flow



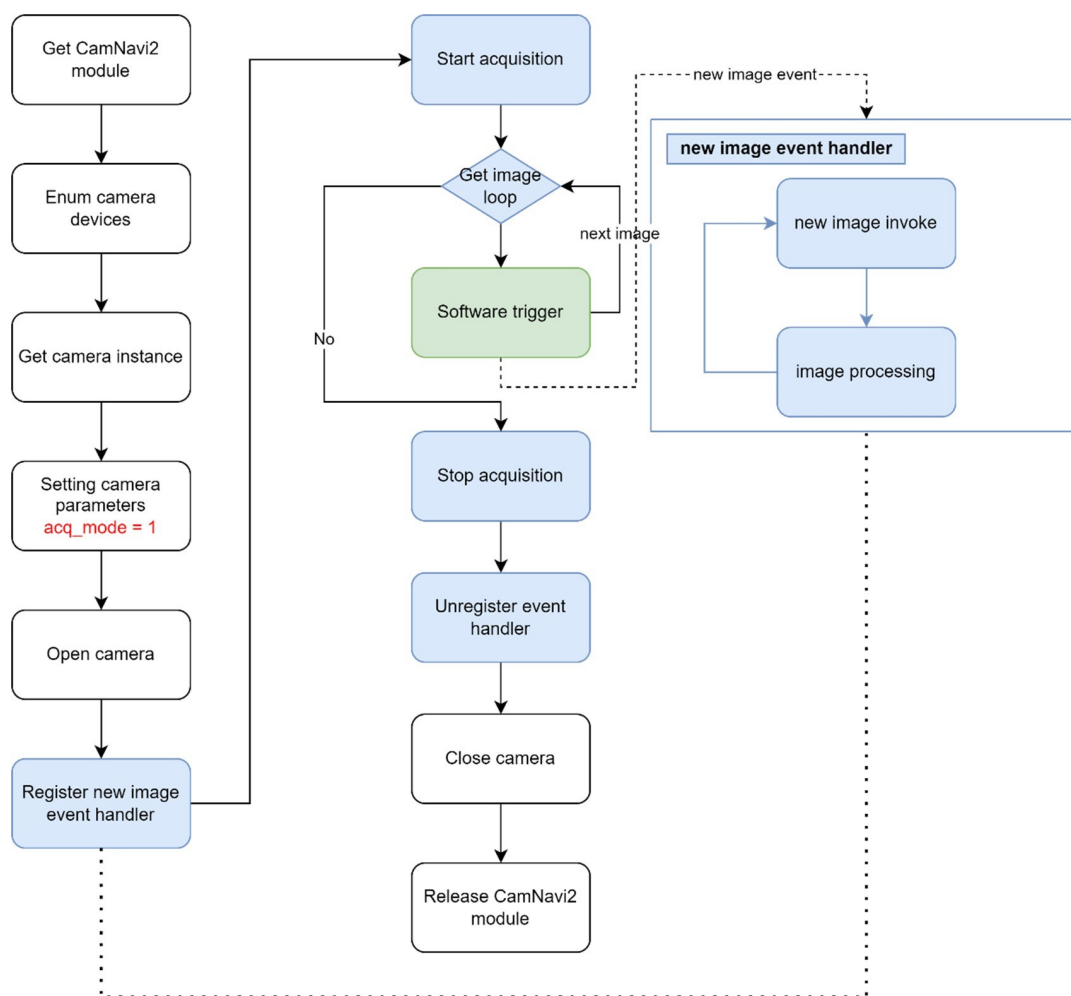
3.2 Image Acquisition



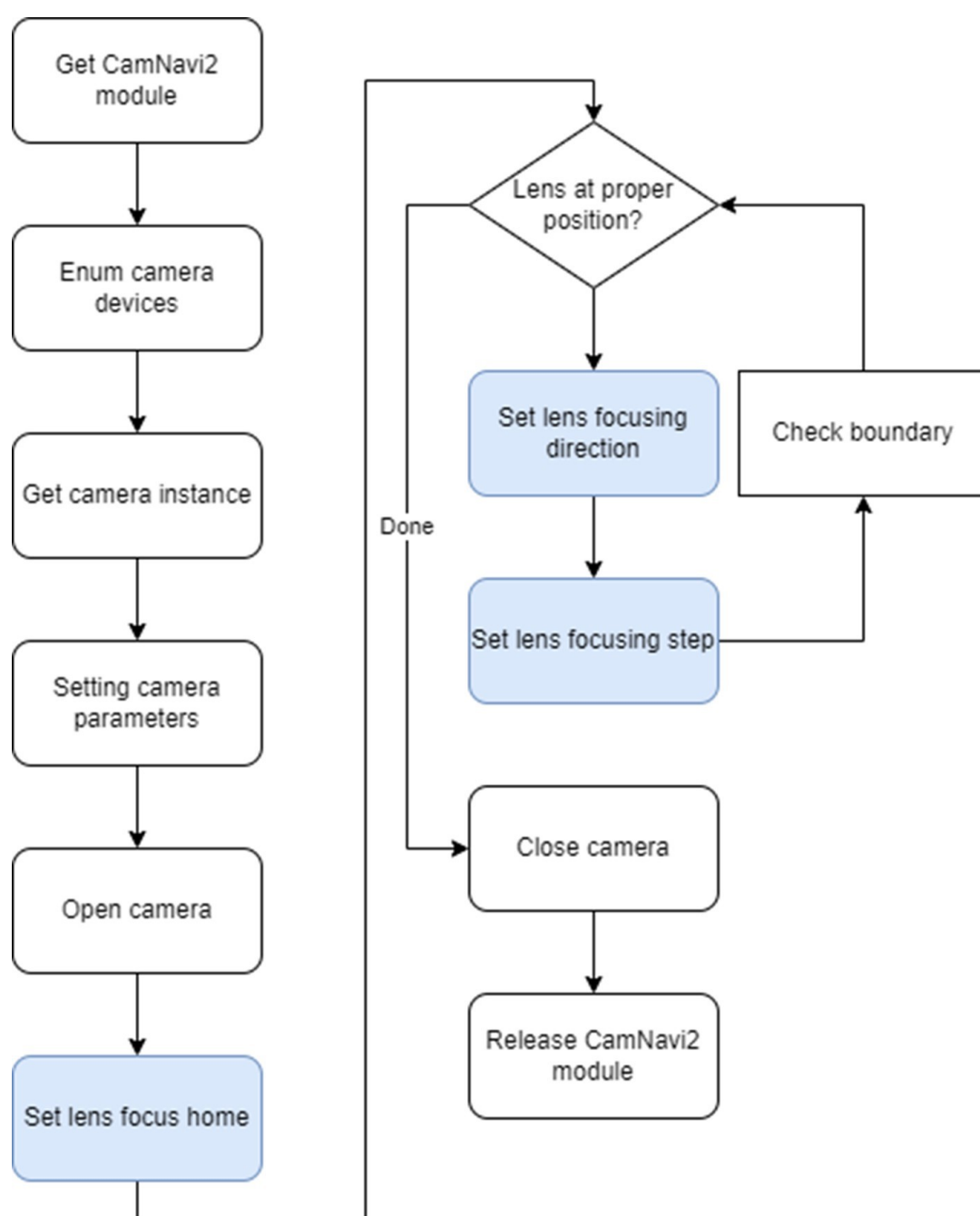
3.3 Image Trigger Flow



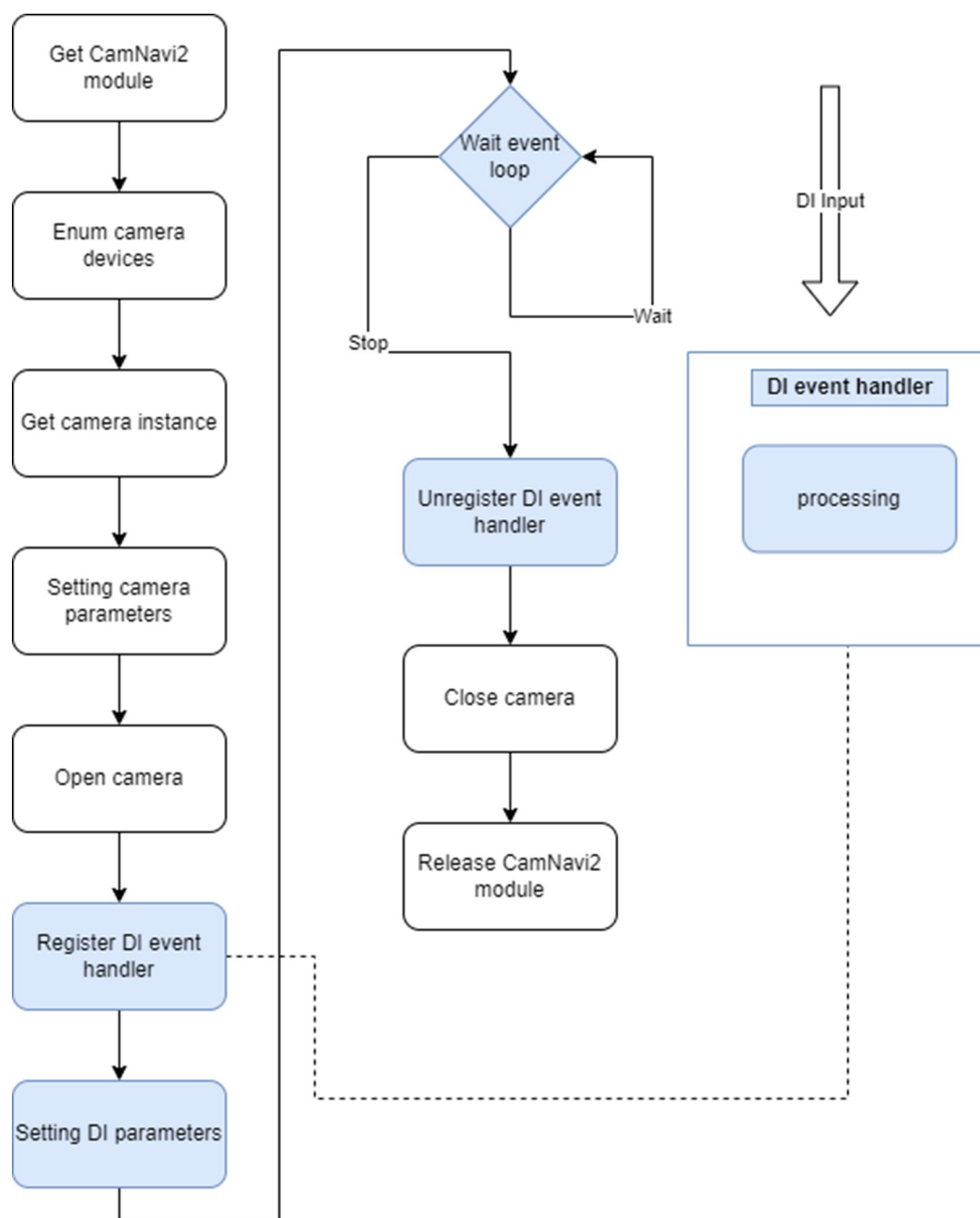
3.3.1 Software Trigger



3.4 Lens Focusing Control



3.4.1 Digital Input flow



Chapter 4

CamNavi2 SDK API
Manual

CamNavi2 SDK provides APIs for the ICAM-300 series. This includes sensor operation, lighting control, board control.

The diagram below shows the SDK components hierarchy:

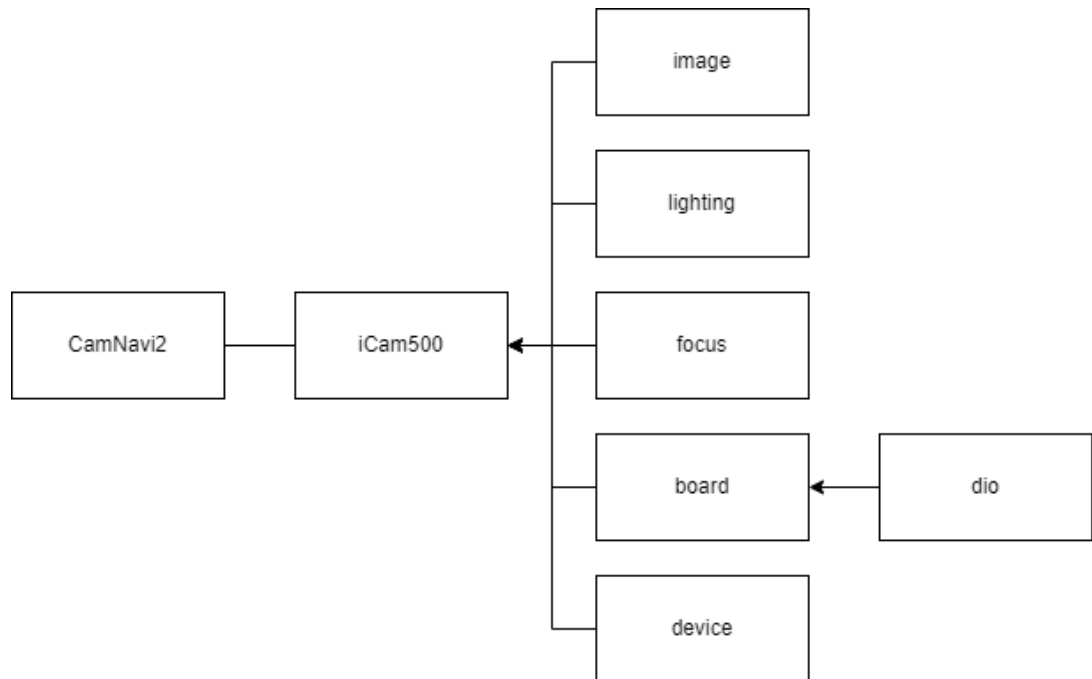


Table 4.1: Module CamNavi2

Function	Description
<code>get_device_by_name()</code>	Use device name to get camera instance.
<code>get_info()</code>	Get BSP information.
<code>get_fw_info()</code>	Get firmware information.
<code>advcam_config_pipeline()</code>	Camera property configuration.
<code>advcam_open()</code>	Open camera instance.
<code>advcam_close()</code>	Close camera instance.
<code>advcam_play()</code>	Start image acquisition process. (Continuous or single shot)
<code>advcam_stop()</code>	Stop image acquisition process.
<code>advcam_snap_image()</code>	Snap an image from streaming. (10ms/image)
<code>advcam_register_new_image_handler()</code>	Register async new image event handler.

Table 4.2: iCam500 Components

Module	Description
image	Configure frame properties.
focus	Lens focusing operation.
lighting	LED lighting control.
dev	ICAM device configuration.(IP configuration)
board	Board IO control (digital input/output, buttons)

4.1 Requirements

4.1.1 Basic

Package contains ICAM-300 BSP:

1. Python 3
2. Python3-gst-1.0
3. Python3-networkmanager

4.2 Module CamNavi2

4.2.1 Class CamNavi2

```
class CamNavi2.CamNavi2(*args, **kwargs)
```

CamNavi2 is a module for management ICAM device.

```
enum_camera_list()
```

Enumerate devices on iCam.

Returns: **dict**

A dict device name and device path mapping.

```
get_device_by_name(name)
```

Get device instance by device name.

Parameters: **name: str**

Device name

Returns: **ABSCamera**

Camera instance. If camera does not find, return None.

```
get_info()
```

General information provides by SDK. Provide BSP information.

Returns: **dict**

Information

```
get_fw_info()
```

Get camera module firmware version.

Returns: **str**

Firmware version string.

Support:

ICAM-300: Yes

advcam_config_pipeline (cam, **pipe_params)

Camera gstreamer pipeline configuration.

Parameters: **params:** dict{str, str}

Set pipeline parameters.

 'format': ['NV12']

 'acq_mode': 0 = Continuous, 1 = Software Trigger

 'width': 640

 'height': 480

 'frame_count': -1 (-1 = continuous, >0 = frame count)

 'icam500=frame-rate': 60

 'timestamp': [0, 1]. Print timestamp on image. 'pipeline_mode':

 ['default', 'simple']. default mode output jpeg image format, simple mode output jpeg image.

See also:

[HYPERLINK \I "iCam500.iCam500.config_pipeline\(\)](#)

advcam_open(cam, frame_count=-1)

Open camera device.

Parameters: **cam:** ABSCamera

Camera instance from get_device_by_name.

frame_count: int

Burst images acquisition.

Returns: **str**

Device name.

None

Camera cannot open.

advcam_close(cam)

Close camera device.

Parameters: **cam:** ABSCamera

Camera instance

Returns: **int**

Close camera status.

 ■ 0: success

 ■ 1: camera instance is not existed.

 ■ 2: camera get an exception.

 ■ 3: camera is no opened.

advcam_get_capability(cam)

Camera capability information.

Returns: **dict**

Camera capability map

advcam_register_new_image_handler(*cam*, *new_image_cb*)

Register a new image event handler.

Event handler sample:

```
def new_sample_cb(sample):buf = smaple.get_buffer()
```

Parameters:

- cam: ABSCamera**
Camera instance
- new_sample_cb**
New image event handler(callback).

Returns: **int**
0: Success.

advcam_play(*cam*)

Start image acquisition step. (Continuous mode or single shot)

Parameters: **cam: ABSCamera**
Camera instance

Returns: **bool**
True: success.

See also:

iCam500.iCam500.play()

advcam_snap_image(*cam*, *timeout*=60)

Snap an image. If timeout reture None.

Parameters: **cam: ABSCamera**
Camera instance

timeout: int
Break waiting loop in timeout second.
unit: second

Returns: **None or [int]**
None: No image or timeout.
array: Image data in JPEG format

advcam_stop(*cam*)

Stop image acquisition procedure.

Returns: **bool**
False: pipeline does not set.
True: Success.

See also:

iCam500.iCam500.stop()

advcam_get_acq_mode(*cam*)

Get acquisition mode property.

Parameters: **cam: ABSCamera**
Camera instance

Returns: **int**
Acquisition mode:

- 0: Free run mode
- 1: Software trigger mode

advcam_set_acq_mode(cam, acq_mode)

Set acquisition mode property.

Parameters: **cam: ABSCamera**
 Camera instance
 acq_mode : int
 Acquisition mode:
 ■ 0: Free run mode
 ■ 1: Software trigger mode

Note! *This API can be set at run-time, but need to re-open (ie. Call `advcam_close()`, then `advcam_open()`) to take effect.*



advcam_get_focus_distance(cam)

Get lens focusing motor distance.

Parameters: **cam: ABSCamera**
 Camera instance

Returns: **int**
 Lens focusing motor distance.

Support:
 ICAM-300: Yes

See also:

iCam500.iCam500.get_focus_distance()

advcam_set_focus_distance(cam, distance)

Set lens focusing motor distance.

Parameters: **cam: ABSCamera**
 Camera instance

distance: **int**
 Lens focusing motor distance.
 init value = 30
 range = [1..300]
 default = 30

Support:
 ICAM-300: Yes

See also:

iCam500.iCam500.get_focus_distance()

advcam_get_lighting_pos(cam)

Get LED lighting location mode.

Parameters: **cam: ABSCamera**
 Camera instance

Returns: **int**
 8 LED trigger location modes. range = [0..7]

Support:
 ICAM-300: Yes

See also:

iCam500.iCam500.get_lighting_pos()

advcam_set_lighting_pos(*cam, pos*)

Set LED lighting location mode.

Parameters: **cam: ABSCamera**

Camera instance

pos: int

Trigger 8 LED lighting location.

init value = 3

range = [0..7]

0: off

Returns: **int**

0: Success

Support:

ICAM-300: Yes

See also:

iCam500.iCam500.set_lighting_pos()

advcam_set_focus_distance(*cam, distance*)

Set LED lighting location mode.

Parameters: **cam: ABSCamera**

Camera instance

distance: int

Lens focusing motor distance.

init value = 0

range = [0..300]

step = 30

See also:

iCam500.iCam500.set_focus_distance()

advcam_get_lighting_intensity(*cam*)

Get LED lighting intensity.

Parameters: **cam: ABSCamera**

Camera instance

Returns: **int**

LED lighting intensity.

Support:

ICAM-300: Yes

See also:

iCam500.iCam500.get_lighting_intensity()

advcam_set_lighting_intensity(*cam, gain*)

Get LED lighting gain.

Parameters: **cam: ABSCamera**

Camera instance

gain: int

LED lighting gain value.

init value = 9

range = [0..25]

step = 1

Returns: **int**

0: Success

Support:

ICAM-300: Yes

See also:

iCam500.iCam500.set_lighting_intensity()

advcam_get_img_brightness(*cam*)

Get image brightness.

Parameters: **cam: ABSCamera**

Camera instance

Returns: **int**

Sensor brightness value.

Return Type:

ICAM-300: Int

See also:

iCam500.iCam500.get_img_brightness()

advcam_set_img_brightness(*cam, brt*)

Set image brightness.

Parameters: **cam: ABSCamera**

Camera instance

brt

brightness value.

Return:

See also:

iCam500.iCam500.set_img_brightness()

advcam_get_img_sharpness(*cam*)

Get camera image sharpness value.

Parameters: **cam: ABSCamera**

Camera instance

Return: **int**

Camera image sharpness value.

advcam_set_img_sharpness(*cam*, *sharpness*)

Set camera image sharpness value.

Parameters: **cam: ABSCamera**

Camera instance

sharpness:

Camera image sharpness value.

Default Range Step

ICAM-300 2 [0..6] 1

Return: **int**

0: Success

advcam_status(*cam*)

Get camera instance status.

Parameters: **cam: ABSCamera**

Camera instance

Return: **CameraMode**

Reference iCam500.CameraMode

advcam_get_output_resolution(*cam*)

Get camera instance status.

Parameters: **cam: ABSCamera**

Camera instance

Return: **tuple(int, int)**

Get (width, height) tuple.

advcam_set_output_resolution(*cam*, *out_res*)

Set frame resolution

Parameters: **cam: ABSCamera**

Camera instance

out_res: tuple(int, int)

Tuple contains (width, height)

Returns: **tuple(int, int)**

Assigned value width, height

Raises: ValueError

width or height not in valid_resolution_list

Note! *This API can be set at run-time, but need to re-open (ie. Call `advcam_close()`, then `advcam_open()`) to take effect.*

**advcam_get_fw_info()**

Get firmware information.

Returns: **str**

Firmware version information.

Support:

ICAM-300: Yes

advcam_query_fw_sku(*cam*)

Get camera sku from firmware.

Parameters: **cam:** **ABSCamera**

Camera instance

Returns: **str**

'0': Mono SKU FW

'1': Color SKU FW

Support:

ICAM-300: Yes

advcam_reboot()

Reboot this camera device.

4.3 Module iCam500

4.3.1 Class iCam500

class iCam500.iCam500(*args, **kwargs)

iCam500 camera class

Control items:

- camera sensor
- image streaming
- lens movement
- strobe
- digital I/O

valid_resolution_list

ICAM-500 supported resolution.

ICAM-500: {2592:1944, 1920:1080, 1408:1080, 1280:960, 640:480, 320:240, 1024:768} type:dict

image

Camera sensor properties control.

type: CImage

focus

Lens focusing motor control.

Lens movement properties.

type: Focus

lighting

Camera LED lighting control.

Strobe control on ICAM-500.

type: Lighting

dev

ICAM-300 device control.

- Device network configuration.
- Firmware update.

board

Device board I/O controls.

- Digital Input/Output. type: dio
- LED on board controls

type: Board

snap_image(timeout=60 s)

Snap an image.

Parameters: **timeout: int**

Wait image time. If timeout return None.

Returns: **[]**

Image data in JPEG format.-

software_trigger()

Invoke software trigger command.

get_pipeline_params(params)**

Get properties value in defined pipeline.

Get width and height of image

params = {'width': 0, 'height': 0}

params = camera.get_pipeline_params(params) for key in params: print("key %s, value %d" % (key, params[key]))

Parameters: **params: dict{str, str}**

Dict of pipeline paramters.

Returns: **dict{str, str}**

config_pipeline(params)**

Configure ICAM-500 gstreamer pipeline in defined format.

Set resolution 640x480 in free run image

params = {'acq_mode':0, 'width':640, 'height':480}

camera.config_pipeline(params)

Parameters: **params: dict{str, str}**

Set pipeline parameters.

'format': ['NV12']

'acq_mode': 0 = Continuous, 1 = Software Trigger

'width': 2592

'height': 1944

'frame_count': -1 (-1 = continuous, >0 = frame count)

'icam500=frame-rate': 60

'timestamp': [0, 1]. Print timestamp on image.

'pipeline_mode': ['default', 'simple']. default mode output jpeg image format, simple mode output raw image.

Returns: **str**

"Config pipeline OK"

Notes

width and height parameter should in valid_resolution_list.

Ex. { 'width': 1920, 'height': 1080 }

get_device_name()

Get device name.

Returns: **str**
Device name.

open(frame_count= -1)

Open ICAM-500 device.

Parameters: **frame_count : int, optional**
■ > 0: streaming stop when get frames up to frame_count.
■ -1: free run streaming.

Returns: **dict{str, str}**
Device name.

close(quiet=0)

Close ICAM device.

Parameters: **quiet: int**
■ 0: default
■ 1: Force turn off lighting.

Returns: **int**
■ 0: Success.
■ 1: Error in exception.

register_new_sample(new_sample_cb)

Register a new image event handler.

Event handler sample:

```
def new_sample_cb(sample): buf = smaple.get_buffer()
```

Parameters: **new_sample_cb**
New image event handler(callback).

Returns: **int**
0: Success.

play()

Start camera acquisition.

Return: **bool**
True: Success

See also:

CamNavi2.CamNavi2.advcam_play()

Wrapped function in CamNavi2.

stop()

Stop image acquisition procedure.

Return: **bool**
False: pipeline does not set
True: Success

See also:

CamNavi2.CamNavi2.advcam_stop()

Wrapped function in CamNavi2.

set_acq_frame_rate(*new_fps*)

Set acquisition frame rate.

Parameters: **new_fps:** int

Set frame rate.

Returns: **int**

Frame rate setting value.

get_lighting_pos()

Get LED lighting location mode.

Return: **int**

8 LED trigger location modes.
range = [0..7]

Support:

ICAM-300: Yes

See also:

lighting.Lighting.selector

Control by lighting instance.

set_lighting_pos(*pos*)

Set LED lighting location mode.

Parameters: **pos:** int

Trigger 8 LED lighting location.
init value = 3
range = [0..7]
0: off

Returns: **int**

0: Success

Support:

ICAM-300: Yes

See also:

lighting.Lighting.selector

Control by lighting instance.

get_lighting_intensity()

Get LED lighting gain.

Returns: **int**

LED lighting intensity value.

Support:

ICAM-300: Yes

See also:

lighting.Lighting.selector

Control by lighting instance.

set_lighting_intensity(*gain*)

Set LED lighting gain.

Parameters: **gain: int**
LED lighting intensity
value. init value = 9
range = [0..25]
step = 1

Returns: **int**
0: Success

Support:
ICAM-300: Yes

See also:

lighting.Lighting.intensity

Control by lighting instance.

get_img_brightness()

Get camera sensor brightness.

Returns: **int**
Sensor brightness value.
ICAM-300: Int

See also:

image.CImage.brightness

Control by lighting instance.

set_img_brightness(*brightness*)

Set camera sensor brightness.

Parameters: **brightness**
Brightness value.

	Default	Range	Step
ICAM-300	0	[-12..12]	1

0: Success

Return: **int**

See also:

image.CImage.brightness

Control by image instance.

get_img_sharpness()

Get camera image sharpness value.

Returns: **int**
Camera image sharpness value.

Support:
ICAM-520: Yes
ICAM-540S: Yes
ICAM-540C: No

See also:

image.CImage.sharpness

Control by image instance.

set_img_brightness(*sharpness*)

Set image sharpness value.

Parameters: **sharpness**

Image sharpness value.

init value = 0

range = [0..100]

step = 1

	Default	Range	Step
ICAM-300	2	[0..6]	1

Return: **int**

0: Success

See also:

image.CImage.sharpness

Control by image instance.

get_img_exposure_time()

Get camera sensor exposure time value.

Returns: **int**

Camera sensor exposure time. (micro sec.)

image.CImage.exposure_time

Control by image instance.

set_img_exposure_time(*exp_time*)

Set camera sensor exposure time value.

Parameters: **exp_time**

Camera sensor exposure value.

Unit: micro sec.

	Default	Range	Step
ICAM-300	33333333	[0.. 9223372036854775807]	1

Return: **int**

new exposure time

See also:

image.CImage.exposure_time

Control by image instance.

get_img_auto_exposure()

Get camera auto exposure function status.

Returns: **int**

Camera auto exposure function status.

■ 1: Enabled

■ 0: Disabled

See also:

image.CImage.auto_exposure

Control by image instance.

set_img_auto_exposure(enable)

Enable camera auto exposure function.

Parameters: **enable**

- 1: Enabled
- 0: Disabled

Returns: **bool**

Camera auto exposure function status.

- 1: Enabled
- 0: Disabled

See also:

image.CImage.auto_exposure

Control by image instance.

get_focus_distance()

Get lens focusing motor move distance.

Return: **int**

Lens focusing motor move distance value.

Support:

ICAM-300: Yes

Step

1

See also:

focus.Focus.distance

Control by focus instance.

set_focus_distance(distance)

Set lens focusing motor distance.

Parameters: **distance: int**

Lens focusing motor distance.
init value = [30]
range = [0..300]

Returns: **int**

Assigned distance value.

Support:

ICAM-300: Yes

See also:

focus.Focus.distance

Control by focus instance.

get_focus_direction()

Get lens focusing motor movement direction.

Return: **int**

- 0: Zoom in
- 1: Zoom out

Support:

ICAM-300: Yes

See also:

focus.Focus.direction

Control by focus instance.

get_focus_direction(*new_direction*)

Set lens focusing motor movement direction.
And invoke set_focus_distance(step) to move lens.

Parameters: **new_direction**

Change motor direction.

- 0: Zoom in
- 1: Zoom out

Support:

ICAM-300: Yes
New direction value.

Returns: **int**

See also:

focus.Focus.direction

Control by focus instance.

get_led_color()

Get board LED color value.

Return: **int**

- 0: off
- 1: green
- 2: orange
- 3: yellow

set_led_color(*new_color*)

Set board LED color.

Parameters: **new_color**

Change LED color

- 0: off
- 1: green
- 2: orange
- 3: yellow

Returns: **int**

New LED color value.

get_status()

Get camera status.
Camera connection and acquisition status.

Returns: **CameraMode**

Reference iCam500.CameraMode

get_output_ress()

Get current frame resolution.

Returns: **tuple(int, int)**

Get width and height tuple.

set_output_res(out_res)

Set frame resolution.

Parameters: **out_res: tuple(int, int)**

Tuple contains (width, height)

Returns: **tuple(int, int)**

Assigned value width, height

Returns: **ValueError**

width or height not in valid_resolution_list

get_roi()

Get frame ROI value.

Returns: **tuple(int, int, int, int)**

Current image frame ROI value. tuple(top, left, right, bottom)

set_roi(roi)

Change frame image ROI value. Range value have to less than resolution value.

Parameters: **roi: tuple(int, int, int, int)**

New image ROI value. tuple(top, left, right, bottom)

Returns: **bool**

True: Success.

set_timestamp_switch(en=1)

Enable timestamp on output image.

Parameters: **en: int**

Parameters:

0: Disable

1: Enable

Returns: **bool**

True: Success.

get_timestamp_switch()

Get timestamp on output image state.

Returns **int**

0: Disable

1: Enable

pos_zero()

Set lens focusing motor to home.

Support:

ICAM-300: Yes

See also:

focus.Focus.pos_zero()

Control by focus instance.

focus_abs_position()

Get lens focusing motor abs position.

Returns: **int**

Motor position.

Support:

ICAM-300: Yes

See also:**focus.Focus.position()**

Control by focus instance.

Rfps()

Get SDK receiver FPS.

Returns: **float**

SDK receiver frame rate

4.3.2 enum CameraMode

class iCam500.CameraMode(value)

Camera device status

Disconnected = 0

Camera disconnected

Connected = 1

Camera connected

Playing = 2

Camera start acquisition

Paused = 3

Camera stop acquisition

4.4 Module Acquisition

class acquisition.Acquisition(camera)

start (frame_count = -1)

Camera start acquisition.

Parameters: **frame_count: int**

-1:

continuous

acquisition

until stop()

> 0: Stream end of frame_count

stop ()

Camera stop acquisition.

property frame_rate

Set/get stream frame rate(fps).

- init value: 60
- range: [1..60]
- step: 1

4.5 Module Image

4.5.1 Class CImage

class image.CImage(camera)

Camera image properties controls.

property brightness

Image brightness setting.

Getter: Read sensor brightness setting.

Setter: Set sensor brightness setting.

ICAM-300

Default0

Range[-12..12]

Step1

Type: int

Raises Value error

Input value out of range.

property iso_mode

Image iso mode setting.

Getter: Read sensor iso mode setting.

Setter: Set sensor iso mode setting.

0: Auto,
1: Deblur

Value: 2: 100, 3: 200, 4: 400, 5: 800,
6:1600, 7: 3200
8: Manual

ICAM-300

Default0

Range[0..8]

Step1

Type: int

Raises ValueError

Input value out of range.

property iso_value

Image iso value in iso_mode: manual(8).

Getter: Read sensor iso value.

Setter: Set sensor iso value.

Value:

ICAM-300

Default800

Range[300..3200]

Step1

Type: int

Raises ValueError

Input value out of range.

property saturation

Color image saturation value.

sku: color

Getter: Read saturation value.

Setter: Set saturation value.

Value:

		Default	Range	Step
	ICAM-300	5	[0..10]	1
Type:	int			
Raises	ValueError			
	Input value out of range.			

property sharpness

Image sharpness setting.

Getter: Read sensor sharpness setting.

Setter: Set sensor sharpness setting.

Value:

		Default	Range	Step
	ICAM-300	2	[0..6]	1
Type:	int			
Raises	ValueError			
	Input value out of range.			

property hue

Color image hue value.

SKU: color

Getter: Read hue value.

Setter: Set hue value.

Value:

		Default	Range	Step
	ICAM-300	128	[0..255]	1
Type:	int			
Raises	ValueError			
	Input value out of range.			

property wb_mode

Color image white-balance mode.

Auto white balance can set mode 3.

In mode 1 (manual-cc-temp), the temp could control by property white_balance.

In mode 2 (manual-rgb-gains), the RGB value control by properties, awb_red, awb_green, awb_blue

SKU: color

Getter: Read white balance mode setting.

Setter: Set white balance mode setting.

0: off,
1: manual-cc-temp
2: manual-rgb-gains
3: auto
4: shade
5: incandescent
6: fluorescent
7: warm-fluorescent
8: daylight
9: cloudy-daylight
10: twilight

Value:

Default

Range

Step

ICAM-520

3

[0..10]

1

Type: int

Raises **ValueError**

Input value out of range.

property awb_mode

Color image auto white balance mode setting.

SKU: color

Getter: Read auto white balance mode setting.

Setter: Set auto white balance mode setting.

Value:

Default

Range

Step

ICAM-300

0

[0..1]
0: Disable
1: Enable

1

Type: int

Raises **ValueError**

Input value out of range.

property white_balance

Set color image WB red channel value. In wb_mode = 1

SKU: color

Getter: Set white-balance value in image.wb_mode = 1

Value:

Default

Range

Step

ICAM-300

6500

[100..10000]

1

Type: int

Raises **ValueError**

Input value out of range.

property awb_rgb

Read color image white balance value. Example: (r, g, b) = camera.image.awb_rgb

SKU: color

Getter: Read auto white balance value.

Type: tuple(red, green, blue)

Raises **ValueError**
Input value out of range.

property awb_red

Set color image WB red channel value. In wb_mode = 2

SKU: color

Setter: Set WB red channel value in image.wb_mode = 2.

Value:

		Default	Range	Step
	ICAM-300	1	[1..255]	1

Type: int

Raises **ValueError**
Input value out of range.

property awb_green

Set color image WB green channel value. In wb_mode = 2

SKU: color

Setter: Set WB green channel value in image.wb_mode = 2.

Value:

		Default	Range	Step
	ICAM-300	1	[1..255]	1

Type: int

Raises **ValueError**
Input value out of range.

property awb_blue

Set color image WB blue channel value. In wb_mode = 2

SKU: color

Setter: Set WB blue channel value in image.wb_mode = 2.

Value:

		Default	Range	Step
	ICAM-300	1	[1..255]	1

Type: int

Raises **ValueError**
Input value out of range.

property width

Image width setting.

Getter: Read image width setting.

Setter: Set image width setting.

Value: ■ init value: 640
 ■ range: [320, 640, 1280, 1408, 1920, 2592]

Type: int

Raises **ValueError**
 Input value out of range.

property height

Image height setting.

Getter: Read image height setting.

Setter: Set image height setting.

Value: ■ init value: 480
 ■ range: [240, 480, 960, 1080, 1080, 1944]

Type: int

Raises **ValueError**
 Input value out of range.

property exposure_time

Camera sensor exposure time setting.

Getter: Read sensor exposure time setting.

Setter: Set sensor exposure time.

Value:

	Default	Range	Step
ICAM-300	33333333	[0.. 9223372036854775807]	1

Type: int

Raises **ValueError**
 Input value out of range.

property auto_exposure

Sensor auto exposure.

Getter: Read sensor auto exposure setting.

Setter: Enable sensor auto exposure setting.

Value: ■ init value: 0
 ■ range: [0, 1]

Type: int

Raises **ValueError**
 Input value out of range.

property pixel_format

Camera sensor pixel format.

Getter: Read sensor pixel format setting.

Setter: Set sensor pixel format.

Value: ■ init value: 'NV12'
■ range: ['NV12']

	ICAM-520	Default	Range
		NV12	['NV12']

Type: int

Raises: **ValueError**
Input value out of range.

property flip_op

Software image flip operation.

Getter: Read image flip state.

Setter: Write image flip property.

Value: ■ init value: 0
■ range: [0, 5]

Type: int

Define: 0: no-flip
1: rotate clockwise 90 degrees
2: rotate 180 degrees
3: rotate counter-clockwise 90 degrees
4: flip horizontally
5: flip vertically

Version: Added after SDK version 1.1.26

4.6 Module Focus

4.6.1 Class Focus

class focus.Focus(camera)

Camera image properties controls.

property distance

Lens focusing motor distance control.

Example:

```
camera.focus.pos_zero() # Set motor to position 0
camera.focus.distance = 30 # Lens move 30
camera.focus.direction = 1 # Motor zoom in 30
camera.focus.direction = 1 # Motor zoom in 30
camera.focus.position() # Get position value is 60
```

Getter: Read motor movement step.

Setter: Set motor movement step.

Value: ■ init value: 0
■ range: [0..300]

Support: ICAM-300: Yes

Type: Int

Raises: **ValueError**
Input value out of range.

property **direction**

Lens focusing motor movement direction control.

Getter: Read motor movement direction.

Setter: Set motor movement direction.

Value:

- 0: Backward
- 1: Foreward

Type: Int

Raises: **ValueError**
Input value out of range.

property **auto_focus**

Lens auto focus.

Getter: Read lens auto-focus mode.

Setter: Set lens auto-focus mode.

Value:

- 0: Disable
- 1: Enable

Type: Int

Raises: **ValueError**
Input value out of range.

position()

Get lens focusing motor position.

Returns **int**
Motor position.

rst_position

Restore lens focusing motor position.

Sk: color

Parameters: steps: int
Lens focusing motor goto this position from home.range: [-512..60]

Raises: **ValueError**
Input value out of range.

pos_zero()

Set lens focusing motor to position 0.

4.7 Module Lighting

4.7.1 Class Lighting

```
class focus.Focus(camera)
```

property selector

LED lighting location mode control. 8 LED trigger location modes.

Getter: Read LED lighting flash location.

Setter: Set LED lighting flash location.

Value:

- init value = 3
- range = [0..7]
- 0: off

Support: ICAM-300: Yes

Type: Int

Raises: **ValueError**
Input value out of range.

property intensity

LED lighting intensity control.

Getter: Read LED lighting intensity value.

Setter: Set LED lighting intensity.

Value:

- init value = 9
- range = [0..25]
- step = 1

Type: Int

Raises: **ValueError**
Input value out of range.

property external_strobe_enable

Enable external lighting strobe signal.

Getter: Read external strobe status.

Setter: Set external strobe lighting status.

Value:

- enable = 1
- disable = 0

Type: Int

Raises: **ValueError**
Input value out of range.

4.8 Module Board

4.8.1 Class Board

```
class board.Board(camera)
    ICAM-500 board I/O control module
    dio
        Digital I/O control instance.
        type: DigitalIO
```

4.9 Module Digitalio

4.9.1 Digital I/O Code Slice

```
# Get camera instance.
camera = cn2.get_device_by_name('iCam500')
# Open camera device
cn2.advcam_open(camera)

# Setting do0 parameters
camera.dio.do0.op_mode = 0 # DO op mode: user output
camera.dio.do0.user_output = 0 # DO low, DI high
camera.dio.do0.reverse = 0

# Setting di0 parameters
camera.dio.di0.invert = 0 # Disable invert
camera.dio.di0.register_event(di_handler) # Register di0 signal handler
camera.dio.di0.trigger_mode = 1 # DI mode
camera.dio.di0.trigger_source = 1 # DI invoke in rising edge
camera.dio.di0.debounce_time = 100

camera.dio.do0.user_output = 1 # DO high, DI low time.sleep(1)
camera.dio.do0.user_output = 0 # DO low, DI high

# Verify DI handler invoke times
assert Counter == 1

# Close camera
cn2.advcam_close(camera)
di event handler

# Digital input signal handler
def di_handler(*arg):
    global Counter
    Counter += 1
```

4.9.2 Class DigitalIO

```
class digitalio.DigitalIO(board)
```

di0

digital input 0.
type: DigitalInput

di1

digital input 1.
type: DigitalInput

do0

digital output 0
type: DigitalOutput

do1

digital output 1. If lighting.external_strobe_enable = 0, do1 function enable.
type: DigitalOutput
support: ICAM-300: Yes

4.9.2.1 Class DigitalInput

```
class digitalio.DigitalInput(board, dev_name)
```

property level

Digital input level state.

Getter: Read input level state.

Value: ■ 1: high
 ■ 0: low

Type: Int

property mode

Enable digital input mode.

Getter: Read mode.

Setter: Enable mode.

Value: ■ 1: Enable DI.
 ■ 0: Off.

Type: Int

Raises: **ValueError**
 Input value out of range.

property source

DI source configuration.

Getter: Read DI source.

Setter: Set DI source.

Value:

- 0: None
- 1: rising edge
- 2: falling edge
- 3: both

Type: Int

Raises: **ValueError**
Input value out of range.

register_event(event_handle)

Register DI trigger event handler.

Example:

```
def di_event():  
    pass
```

Parameters: **event_handler:**
Event handler function. Reference di_event example.

4.9.2.2 Class DigitalOutput

```
class digitalio.DigitalOutput(board, dev_name)
```

property user_output

User set output level. This property works on op_mode is 0

Getter: Read output signal level.

Setter: Set output signal level.

Value:

- 0: low level
- 1: high level

Raises: **ValueError**
Input value out of range.

4.10 Module Device

4.10.1 Class Network

```
class device.Network
```

```
    info()
```

Get ICAM-500 device network configuration

Returns: **tuple(str, str, str, str, str)**

Input value out of range.

1.mode : 'dhcp' or 'static'

2.IP: IPv4 address

3.netmask: IPv4 netmask

4.gateway: IPv4 gateway address

5.MAC address: ICAM-500 MAC address

```
config(mode='dhcp', ip=None, netmask=None, gateway=None)
```

Configure ICAM-500 network

```
    info()
```

Get ICAM-500 device network configuration

Parameters: **mode: str**

- 'dhcp': Set network in DHCP mode. ip, netmask, gate- way must set None.

- 'static': Set a static IPv4 address.

ip: str

IPv4 address. ex. '192.168.0.100'

netmask: str

IPv4 netmask. ex. '255.255.255.0'

gateway: str

IPv4 gateway. ex. '192.168.0.1'

Raises: **AttribureError**

If mode is "dhcp", ip/netmask/gateway must set None.

4.10.2 Class Firmware

```
class device.Firmware(camera)
```

```
    info()
```

Read firmware version

Support: ICAM-300: Yes

Returns: **str**

Firmware version. ex. '1.7'

4.11 Exceptions

iCam500 properties value out of range would raise ValueError exceptions.

```
try:
    camera.lighting.selector = 8
except ValueError:
    print("lighting selector out of range")
```

Table 4.3: API Support Table

Module	API	ICAM-300	ICAM-540S	ICAM-540C
CamNavi2	enum_camera_list()	Yes	Yes	Yes
	get_device_by_name()	Yes	Yes	Yes
	get_info()	Yes	Yes	Yes
	get_fw_info()	Yes	Yes	No
	CamNavi2.advcam_config_pipeline()	Yes	Yes	Yes
	advcam_open()	Yes	Yes	Yes
	advcam_close()	Yes	Yes	Yes
	advcam_register_new_image_handler()	Yes	Yes	Yes
	advcam_play()	Yes	Yes	Yes
	advcam_snap_image()	Yes	Yes	Yes
	advcam_stop()	Yes	Yes	Yes
	advcam_get_acq_mode()	Yes	Yes	Yes
	advcam_set_acq_mode()	Yes	Yes	Yes
	advcam_get_focus_distance()	Yes	Yes	No
	advcam_set_focus_distance()	Yes	Yes	No
	advcam_get_lighting_pos()	Yes	Yes	No
	advcam_set_lighting_pos()	Yes	Yes	No
	advcam_get_lighting_gain()	Yes	Yes	No
	advcam_set_lighting_gain()	Yes	Yes	No
	advcam_get_lighting_strobe_enable()	No	Yes	No
	advcam_set_lighting_strobe_enable()	No	Yes	No
	advcam_get_img_brightness()	Yes	Yes	Yes
	advcam_set_img_brightness()	Yes	Yes	Yes
	advcam_get_img_gain()	No	Yes	Yes
	advcam_set_img_gain()	No	Yes	Yes
	advcam_get_img_sharpness()	Yes	Yes	Yes
	advcam_set_img_sharpness()	Yes	Yes	Yes
	advcam_status()	Yes	Yes	Yes
	advcam_get_output_resolution()	Yes	Yes	Yes
	advcam_set_output_resolution()	Yes	Yes	Yes
	advcam_get_fw_info()	Yes	Yes	No
	advcam_query_fw_sku()	Yes	No	No
	advcam_reboot()	Yes	Yes	Yes
	snap_image()	Yes	Yes	Yes
	software_trigger()	Yes	Yes	Yes
	hw_trigger_delay	No	Yes	Yes
	hw_trigger_exp_src	No	Yes	No
	config_pipeline()	Yes	Yes	Yes
	get_device_name()	Yes	Yes	Yes

Table 4.3: API Support Table

	open()	Yes	Yes	Yes
	close()	Yes	Yes	Yes
	register_new_sample()	Yes	Yes	Yes
	play()	Yes	Yes	Yes
	stop()	Yes	Yes	Yes
	set_acq_frame_rate()	Yes	Yes	Yes
	get_lighting_pos()	Yes	Yes	No
	set_lighting_pos()	Yes	Yes	No
	get_lighting_gain()	Yes	Yes	No
	set_lighting_gain()	Yes	Yes	No
	get_lighting_strobe_enable()	No	Yes	No
	set_lighting_strobe_enable()	No	Yes	No
	get_img_brightness()	Yes	Yes	Yes
	set_img_brightness()	Yes	Yes	Yes
	get_img_gain()	No	Yes	Yes
	set_img_gain()	No	Yes	Yes
	get_img_sharpness()	Yes	Yes	No
iCam500	set_img_sharpness()	Yes	Yes	No
	get_img_exposure_time()	Yes	Yes	Yes
	set_img_exposure_time()	Yes	Yes	Yes
	get_img_auto_exposure()	Yes	Yes	Yes
	set_img_auto_exposure()	Yes	Yes	Yes
	get_img_auto_exposure_range()	No	Yes	Yes
	set_img_auto_exposure_range()	No	Yes	Yes
	get_img_auto_gain()	No	Yes	Yes
	set_img_auto_gain()	No	Yes	Yes
	get_img_auto_gain_range()	No	Yes	Yes
	set_img_auto_gain_range()	No	Yes	Yes
	get_focus_distance()	Yes	Yes	No
	set_focus_distance()	Yes	Yes	No
	get_focus_direction()	Yes	Yes	No
	set_focus_direction()	Yes	Yes	No
	get_led_color()	Yes	Yes	Yes
	set_led_color()	Yes	Yes	Yes
	get_status()	Yes	Yes	Yes
	get_output_res()	Yes	Yes	Yes
	set_output_res()	Yes	Yes	Yes
	get_roi()	Yes	Yes	Yes
	set_roi()	Yes	Yes	Yes
	set_timestamp_switch()	Yes	Yes	Yes
	get_timestamp_switch()	Yes	Yes	Yes
	pos_zero()	Yes	Yes	No
iCam500	focus_abs_position()	Yes	Yes	No
	Rfps()	Yes	Yes	Yes
acquisition	line0_debounce_time	No	Yes	No

Table 4.3: API Support Table

image	brightness	Yes	Yes	Yes
	gain	No	Yes	Yes
	saturation_op	No	Yes	No
	saturation	Yes	Yes	Yes
	sharpness	Yes	Yes	No
	gamma	No	Yes	Yes
	hue	Yes	Yes	Yes
	awb_op	No	Yes	Yes
	awb_mode	Yes	Yes	Yes
	awb_rgb	Yes	Yes	Yes
	awb_red	Yes	Yes	Yes
	awb_green	Yes	Yes	Yes
	awb_blue	Yes	Yes	Yes
	color_trans_op	No	Yes	No
	color_trans_00	No	Yes	No
	color_trans_01	No	Yes	No
	color_trans_02	No	Yes	No
	color_trans_10	No	Yes	No
	color_trans_11	No	No	Yes
	color_trans_12	No	No	Yes
	color_trans_20	No	No	Yes
	color_trans_21	No	No	Yes
	color_trans_22	No	No	Yes
	width	Yes	Yes	Yes
	height	Yes	Yes	Yes
	exposure_time	Yes	Yes	Yes
	auto_exposure_value	No	Yes	Yes
	auto_exposure	Yes	Yes	Yes
	auto_exposure_range	No	Yes	Yes
	auto_gain_value	No	Yes	Yes
	auto_gain	No	Yes	Yes
	auto_gain_range	No	Yes	Yes
	pixel_format	Yes	Yes	Yes
	x_mirror	No	Yes	Yes
	y_mirror	No	No	Yes
	flip_op	Yes	Yes	Yes
focus	distance	Yes	Yes	Yes
	direction	Yes	Yes	Yes
	position()	Yes	Yes	Yes
	rst_position()	Yes	Yes	Yes
	pos_zero()	Yes	Yes	Yes
lighting	selector	Yes	Yes	No
	gain	Yes	Yes	No
	strobe_enable	No	Yes	No
DigitalInput	level	Yes	Yes	Yes
	mode	Yes	Yes	Yes
	source	Yes	Yes	Yes
	debounce_time	No	Yes	Yes
	debounce_mode	No	No	No
	register_event()	Yes	Yes	Yes

Table 4.3: API Support Table

DigitalOutput	user_output	Yes	Yes	Yes
	op_mode	No	No	No
	reverse	No	No	No
	delay_time	No	No	No
Network	info()	Yes	Yes	Yes
	config()	Yes	Yes	Yes
Firmware	info()	Yes	Yes	No

Chapter 5

CamNavi2 SDK Python
Examples

5.1 Grab Frame Continuous

We recommend installing python3-opencv for image processing in the samples. Grab image frames using the callback function.

Register image grabber callback function:

```
CamNavi2.advcam_register_new_image_handler(camera,  
new_image_handler) or camera.register_new_sample(new_im-  
age_handler)
```

Callback function snippet:

```
def new_image_handler(sample): is callback funtion. sample  
parameter is image data.
```

Image conversion snippet:

```
def gst_to_opencv(sample): this function decode image to  
cv2:Mat for- mat
```

```
#!/usr/bin/env python3  
  
import cv2  
import numpy  
import time  
  
from CamNavi2 import CamNavi2  
  
image_arr = None  
icam_color = 0  
  
def gst_to_opencv(sample):  
    # Extract gray jpeg image from sample  
    buf = sample.get_buffer()  
  
    buffer = buf.extract_dup(0, buf.get_size())  
    arr = numpy.frombuffer(buffer, dtype=numpy.uint8)  
    if icam_color == 1:  
        im = cv2.imdecode(arr, cv2.IMREAD_COLOR)  
    else:  
        im = cv2.imdecode(arr, cv2.IMREAD_GRAYSCALE)  
  
    return im  
  
# Handle new image income  
def new_image_handler(sample):  
    global image_arr  
    if sample is None:  
        return  
  
    # Convert sample  
    arr = gst_to_opencv(sample)  
    image_arr = arr  
  
if __name__ == '__main__':  
    try:  
        cn2 = CamNavi2.CamNavi2()  
    except:  
        cn2 = CamNavi2()
```

```

# Config camera resolution
# iCAM-500 valid resolution list: 1280x960, 640x480, 320x240
width = 640
height = 480

camera_dict = cn2.enum_camera_list()
print("\nEnum. available camera list: ", camera_dict)

sdk_info = cn2.get_info()
print("\nGeneric SDK information: ", sdk_info)

# Get camera instance.
camera = cn2.get_device_by_name('iCam500')

# Get camera sku to check determine image channel
# 0 mono, 1 color
icam_color = int(cn2.advcam_query_fw_sku(camera))
print('icam-500 sku: %d' % (icam_color))

# Set camera resolution and acquisition method block.
# init pipeline for a camera module before open it
pipe_params = {'acq_mode':0, 'width':width, 'height': height,
'enable_infer':0}
if icam_color == 1:
    pipe_params['format'] = 'YUY2'
# pipe_params = {} # using default settings
pipe_str = cn2.advcam_config_pipeline(camera,**pipe_params)

# Open camera device, and get continuous image
#cn2.advcam_open(camera)
# Open camera device, and declare 100 image frame count
cn2.advcam_open(camera, 100)

# Register new image handler
cn2.advcam_register_new_image_handler(camera,new_image_handler)

# Start image streaming.
cn2.advcam_play(camera)
cn2.advcam_set_img_brightness(camera, 50)
cn2.advcam_set_img_gain(camera, 10)

```

```

while True:
    try:
        if image_arr is not None:
            # Display image if image_arr has data cv2.imshow
            ("appsink", image_arr) lastKey = cv2.waitKey(10)
            if lastKey == 27: cv2.destroyAllWindows()
            break
        else:
            break

    except KeyboardInterrupt:
        break

# unregister new image handler
cn2.advcam_register_new_image_handler(camera, None)
# camera.close()
cn2.advcam_close(camera)

```

5.2 Grab Frames by Software Trigger

Use software to trigger image acquisition.

Image grabbing procedure:

Same as Grab Frame Continuous describe.

Assign acq_mode:

acq_mode assign 1 for software trigger mode. If assign 2 is hardware trigger.

Trigger acquisition:

Use `camera.software_trigger()` to trigger image acquisition.

```

#!/usr/bin/env python3

import cv2
import numpy
import time

from CamNavi2 import CamNavi2

image_arr = None
icam_color = 0

'''
Image acquisition in software trigger method.
Press "Enter" key to invoke software trigger.
Press "Esc" key to quit this program
'''

```

```

def gst_to_opencv(sample):
    # Extract gray jpeg image from sample
    buf = sample.get_buffer()

    buffer = buf.extract_dup(0, buf.get_size())
    arr = numpy.frombuffer(buffer, dtype=numpy.uint8)
    if icam_color == 1:
        im = cv2.imdecode(arr, cv2.IMREAD_COLOR)
    else:
        im = cv2.imdecode(arr, cv2.IMREAD_GRAYSCALE)

    return im

# Handle new image income
def new_image_handler(sample):
    global image_arr
    if sample is None:
        return

    # Convert sample
    arr = gst_to_opencv(sample)
    image_arr = arr
    #cv2.imwrite('./sample.jpg', arr) # write image

if __name__ == '__main__':
    try:
        cn2 = CamNavi2.CamNavi2()
    except:
        cn2 = CamNavi2()

    # Config camera resolution
    # iCAM-500 valid resolution list: 1280x960, 640x480, 320x240
    width = 640
    height = 480

    camera_dict = cn2.enum_camera_list()
    print("\nEnum. available camera list: ", camera_dict)

    sdk_info = cn2.get_info()
    print("\nGeneric SDK information: ", sdk_info)

    # Get camera instance.
    camera = cn2.get_device_by_name('iCam500')

    # Get camera sku to check determine image channel
    # 0 mono, 1 color
    icam_color = int(cn2.advcam_query_fw_sku(camera))
    print('icam-500 sku: %d' % (icam_color))

```

```

# Set camera resolution and acquisition method block.
# init pipeline for a camera module before open it
# acq_mode: 1 software trigger mode
pipe_params = {'acq_mode':1, 'width':width, 'height': height,
'enable_infer':0}
if icam_color == 1:
    pipe_params['format'] = 'YUY2'
# pipe_params = {} # using default settings
pipe_str = cn2.advcam_config_pipeline(camera, **pipe_params)

# Open camera device, and get continuous image
#cn2.advcam_open(camera)
# Open camera device, and declare 100 image frame count
cn2.advcam_open(camera, 10)

# Register new image handler
cn2.advcam_register_new_image_handler(camera,
new_image_handler)

# Start image streaming.
cn2.advcam_play(camera)
cn2.advcam_set_img_brightness(camera, 50)
# cn2.advcam_set_img_gain(camera, 10)
exp_time = camera.image.exposure_time * 100 # us
exp_time /= 1000000 # sec.
print('exposure time: ', exp_time , ' sec')

# Fetch first image
camera.software_trigger()
if exp_time < 0.5:
    time.sleep(0.5)
while True:
    try:
        if image_arr is not None:
            # Display image if image_arr has data cv2.imshow
            ("appsink", image_arr) lastKey = cv2.waitKey(10)
            if lastKey == 27: # Esc cv2.destroyAllWindows()
                break
            elif lastKey == 10: # Enter camera.software_
                trigger()
        else:
            break
    except KeyboardInterrupt:
        break

# unregister new image handler
cn2.advcam_register_new_image_handler(camera, None)
# camera.close()
cn2.advcam_close(camera)

```

5.3 Digital Input Processing

Set DI mode and source:

camera.dio.di0.mode assign 1 to enable DI interrupt.
camera.dio.di0.source select DI invoke signal.

Register DI callback function:

camera.dio.di0.register_event(di_handler)

Callback function snippet:

```
def di_handler(*arg):
```

```
#!/usr/bin/env python3

import time

from CamNavi2 import CamNavi2

'''
Pin connection: do0-----di0
'''

Counter = 0
# Digital input signal handler
def di_handler(*arg):
    global Counter
    Counter += 1

if __name__ == '__main__':
    try:
        cn2 = CamNavi2.CamNavi2()
    except:
        cn2 = CamNavi2()

    # Config camera resolution
    # iCAM-500 valid resolution list: 1280x960, 640x480, 320x240
    width = 640
    height = 480

    camera_dict = cn2.enum_camera_list()
    print("\nEnum. available camera list: ", camera_dict)
    sdk_info = cn2.get_info()
    print("\nGeneric SDK information: ", sdk_info)

    # Get camera instance.
    camera = cn2.get_device_by_name('iCam500')

    # Set camera resolution and acquisition method block.
    # init pipeline for a camera module before open it
    # acq_mode: 1 software trigger mode
    pipe_params = {'acq_mode':1, 'width':width, 'height': height,
                  'enable_infer':0}
    # pipe_params = {} # using default settings
    pipe_str = cn2.advcam_config_pipeline(camera, **pipe_params)
```

```

# Open camera device
cn2.advcam_open(camera)

# Setting do0 parameters
camera.dio.do0.op_mode = 0 # DO op mode: user output
camera.dio.do0.user_output = 0 # DO low, DI high
camera.dio.do0.reverse = 0

# Setting di0 parameters
camera.dio.di0.invert = 0 # Disable invert
camera.dio.di0.register_event(di_handler) # Register di0 signal handler
camera.dio.di0.mode = 1 # DI mode
camera.dio.di0.source = 1 # DI invoke in rising edge
camera.dio.di0.debounce_time = 100

time.sleep(1)
times = 100

# DO simulate pulse to produce DI
for i in range(0, times):
    camera.dio.do0.user_output = 1 # DO high, DI low time.sleep(1)
    camera.dio.do0.user_output = 0 # DO low, DI high
    time.sleep(1)

# Verify DI handler invoke times
assert Counter == times

# Close camera
cn2.advcam_close(camera)

```

5.4 Lens Focusing Control

Press “Enter” to grab image.

Press “+” to move lens forward.

Press “-” to move lens bakeward.

Press “Esc” to quit program.

```
#!/usr/bin/env python3

import cv2
import numpy
import time

from CamNavi2 import CamNavi2

image_arr = None
icam_color = 0

def gst_to_opencv(sample):
    # Extract gray jpeg image from sample
    buf = sample.get_buffer()

    buffer = buf.extract_dup(0, buf.get_size())
    arr = numpy.frombuffer(buffer, dtype=numpy.uint8)
    if icam_color == 1:
        im = cv2.imdecode(arr, cv2.IMREAD_COLOR)
    else:
        im = cv2.imdecode(arr, cv2.IMREAD_GRAYSCALE)

    return im

# Handle new image income
def new_image_handler(sample):
    global image_arr

    if sample is None:
        return

    # Convert sample
    arr = gst_to_opencv(sample)
    image_arr = arr

if __name__ == '__main__':
    try:
        cn2 = CamNavi2.CamNavi2()
    except:
        cn2 = CamNavi2()

    # Config camera resolution
    # iCAM-500 valid resolution list: 1280x960, 640x480, 320x240
    width = 640
    height = 480

    camera_dict = cn2.enum_camera_list()
    print("\nEnum. available camera list: ", camera_dict)
```

```

sdk_info = cn2.get_info()
print("\nGeneric SDK information: ", sdk_info)

# Get camera instance.
camera = cn2.get_device_by_name('iCam500')#

# Get camera sku to check determine image channel
# 0 mono, 1 color
icam_color = int(cn2.advcam_query_fw_sku(camera))
print('icam-500 sku: %d' % (icam_color))

# Set camera resolution and acquisition method block.
# init pipeline for a camera module before open it
# acq_mode: 1 software trigger mode
pipe_params = {'acq_mode':1, 'width':width, 'height': height,
'enable_infer':0}
if icam_color == 1:
    pipe_params['format'] = 'YUY2'
# pipe_params = {} # using default settings
pipe_str = cn2.advcam_config_pipeline(camera,**pipe_params)

# Open camera device, and get continuous image
#cn2.advcam_open(camera)
# Open camera device, and declare 100 image frame count
cn2.advcam_open(camera, 10)

# Register new image handler
cn2.advcam_register_new_image_handler(camera,
new_image_handler)

# Start image streaming.
cn2.advcam_play(camera)
cn2.advcam_set_img_brightness(camera, 50)
cn2.advcam_set_img_gain(camera, 10)

# Set lens focusing motor to 0
camera.focus.pos_zero()

```

```

# Get first image
camera.software_trigger()
while True:
    try:
        if image_arr is not None:
            # Display image if image_arr has data cv2.imshow
            ("appsink", image_arr) lastKey = cv2.waitKey(10)
            if lastKey == 27: # Esc cv2.destroyAllWindows()
                break
            elif lastKey == 10: # Enter camera.software_trigger()
            elif lastKey == 43: # '+' camera.focus.direction =
                0 # lens focusing motor forward
                try:
                    camera.focus.distance = 30
                    print("lens motor posistion: ",camera.focus.
                        position())
                except ValueError:
                    print("lens position out of index") camera.
                        software_trigger()
            elif lastKey == 45: # '-'
                camera.focus.direction = 1 # lens focusing
                motor backward
                try:
                    camera.focus.distance = 30
                    print("lens motor posistion: ",camera.focus.
                        position())
                except ValueError:
                    print("lens position out of index")
                    camera software_trigger()
        else:
            break
    except KeyboardInterrupt:
        break

# unregister new image handler
cn2.advcam_register_new_image_handler(camera, None)
# camera.close()
cn2.advcam_close(camera)

```

5.5 Lighting Control

Controls 8 LED lighting positions.

```
#!/usr/bin/env python3

import time

from CamNavi2 import CamNavi2

if __name__ == '__main__':
    try:
        cn2 = CamNavi2.CamNavi2()
    except:
        cn2 = CamNavi2()

    # Config camera resolution
    # iCAM-500 valid resolution list: 1280x960, 640x480, 320x240
    width = 640
    height = 480

    camera_dict = cn2.enum_camera_list()
    print("\nEnum. available camera list: ", camera_dict)

    sdk_info = cn2.get_info()
    print("\nGeneric SDK information: ", sdk_info)

    # Get camera instance.
    camera = cn2.get_device_by_name('iCam500')#

    # Set camera resolution and acquisition method block.
    # init pipeline for a camera module before open it
    # acq_mode: 1 software trigger mode
    pipe_params = {'acq_mode':1, 'width':width, 'height': height,
                  'enable_infer':0}
    # pipe_params = {} # using default settings
    pipe_str = cn2.advcam_config_pipeline(camera,**pipe_params)

    # Open camera device
    #cn2.advcam_open(camera)
    camera.acquisition.strobe0_op = 0
    # camera.lighting.strobe_enable = 0
    camera.lighting.gain = 50
    # Changing lighting 8 position mode
    for pos in range(0, 8):
        camera.lighting.selector = pos
        time.sleep(1)

    # Close lighting
    camera.lighting.selector = 0

    # Close camera
    cn2.advcam_close(camera)
```

5.6 Get Image from Video10

Use opencv to access video10 video stream.

```
#!/usr/bin/env python3
#
# Before running this sample
# Please ensure
# 1. you have the necessary package installed.
# ins_py36_dep.sh
# 2. icam-500 is at playing status
#
# when icam-500 is at playing at >= 5fps, there is another rtsp
stream available at port 8550
# rtsp://<ip address>:8550/video
#
# version:
# 20230116: Added save png and save raw image methods.

import cv2
import time

def show_cam():
    # icam-500 is playing and output frames to /dev/video10
    cam = cv2.VideoCapture(10)
    cam.set(cv2.CAP_PROP_FORMAT, -1)
    while True and cam.isOpened():
        ret, img = cam.read()
        if ret:
            cv2.imshow('icam-500', img)
            key = cv2.waitKey(1)
            if key == 27: # esc key
                break
            elif key == ord('s'): # press 's' to save image
                cv2.imwrite('./video10.png', img)

        else:
            print('failed to capture an image')
            time.sleep(0.2)

    cv2.destroyAllWindows()
    print('end program')

if __name__ == '__main__':
    show_cam()
```

www.advantech.com

Please verify specifications before quoting. This guide is intended for reference purposes only.

All product specifications are subject to change without notice.

No part of this publication may be reproduced in any form or by any means, such as electronically, by photocopying, recording, or otherwise, without prior written permission from the publisher.

All brand and product names are trademarks or registered trademarks of their respective companies.

© Advantech Co., Ltd. 2026